Tài liệu kỹ thuật: VIB Internal Knowledge Assistant

COS40005 Computing Technology Project A · Swinburne University of Technology ·
Oct 31, 2026 · @Truong Ngoc Gia Hieu

Tài liệu này đặc tả thiết kế kỹ thuật của VIB Internal Knowledge Assistant, một bot hỏi–đáp trên Slack trả lời câu hỏi của nhân viên từ nội dung Confluence và dẫn nguồn kèm theo. Tài liệu ghi lại kiến trúc, hành vi khi vận hành, mô hình bảo mật và kế hoạch đánh giá, đồng thời đối chiếu các lựa chọn thiết kế với hai công trình đã công bố (Boonsri & Buranasaksee 2025; Mohammed & Aref n.d.).

1. Giới thiệu và phạm vi

1.1 Mục đích

Nhân viên tại tổ chức tài trợ (VIB) lưu tri thức về chính sách và sản phẩm trong Confluence, nơi việc tra cứu tốn nhiều thời gian. Dự án *Internal Assistant for Enterprise Knowledge (Slack App Bot)* cho phép nhân viên đặt câu hỏi trong Slack và nhận câu trả lời ngắn, kèm liên kết đến các trang Confluence đã dùng. Hệ thống áp dụng phương pháp sinh văn bản có truy xuất bổ trợ (retrieval-augmented generation, RAG): các đoạn liên quan được truy xuất trước, sau đó mô hình ngôn ngữ lớn (large language model, LLM) chỉ trả lời dựa trên các đoạn đó.

1.2 Phạm vi

Công việc được giao theo hai bản phát hành, như quy định trong Project Charter.

Bản phát hành	Bao gồm	Loại trừ hoặc hoãn lại
Alpha	Ứng dụng Slack Bolt ở chế độ Socket Mode; danh sách kênh được duyệt; kiểm tra danh tính người dùng đã đăng ký; nạp dữ liệu HTML sang Markdown từ một space Confluence mẫu; cắt đoạn, embedding và tìm kiếm vector; trả lời có dẫn nguồn bằng tiếng Anh và tiếng Việt; LLM chạy cục bộ	Xếp hạng lại, kiểm soát truy cập theo tầng, audit log, bộ nhớ hội thoại, đồng bộ gia tăng, bộ xử lý Word/PDF/Excel/OCR, nút phản hồi

Bản phát hành	Bao gồm	Loại trừ hoặc hoãn lại
Beta	Các mục hoãn lại ở trên	Dữ liệu thật (bản mẫu chỉ dùng space VIB-Mock-Knowledge)

Hai biến thể triển khai dùng chung một thiết kế. Project A dùng thành phần miễn phí, chạy cục bộ (Ollama với Llama-3-8B hoặc Phi-3). Project B nhắm đến buổi trình diễn VIB trên AWS, dùng Amazon Bedrock làm nhà cung cấp LLM.

1.3 Trạng thái tài liệu

Các mục gắn nhãn *giả định thiết kế* hoặc *đề xuất* do nhóm tác giả tài liệu này đưa ra và chưa được thống nhất với khách hàng. Mục 9 liệt kê các mục đó.

1.4 Cấu trúc

Mục 2 tổng quan hai công trình. Các mục 3 đến 7 đặc tả yêu cầu, kiến trúc, luồng vận hành, bảo mật và xử lý dữ liệu. Mục 8 trình bày cách đánh giá hệ thống. Mục 9 ghi nhận rủi ro và các quyết định còn mở.

2. Cơ sở lý thuyết và các nghiên cứu liên quan

Mục này tóm tắt hai công trình làm cơ sở cho thiết kế, nêu đóng góp của từng công trình và những gì còn bỏ ngỏ.

2.1 Kiến trúc tham chiếu cho tác nhân hội thoại

Mohammed và Aref (n.d.) mô tả một kiến trúc chatbot tổng quát gồm ba thành phần cốt lõi. Hiểu ngôn ngữ tự nhiên (Natural Language Understanding, NLU) biến một phát ngôn thành biểu diễn có cấu trúc thông qua phát hiện chủ đề, phân tích ý định và liên kết thực thể. Bộ quản lý hội thoại (Dialogue Manager, DM) chọn chiến lược phản hồi, mà các tác giả sắp theo thứ tự dựa trên luật, dựa trên tri thức, dựa trên truy xuất và sinh tự do. Sinh ngôn ngữ tự nhiên (Natural Language Generation, NLG) lọc các câu trả lời ứng viên về tính mạch lạc, xếp hạng chúng và trả về văn bản hoặc giọng nói. Một mô-đun theo dõi ngữ cảnh (context tracking) lấy các phát ngôn gần nhất từ kho lịch sử trò chuyện để giải quyết các tham chiếu đến lượt trước. Các tác giả xem DM là thành phần quan trọng nhất vì chính thành phần này tạo nên sự khác biệt giữa các hệ thống. Nghiên cứu điển hình trong bài cho biết bot Messenger của KLM làm tương tác với khách hàng tăng 40%, nhưng bài không có đánh giá đối chứng và ra đời trước các hệ thống dựa trên LLM và RAG.

Bài báo hữu ích ở chỗ cung cấp bộ thuật ngữ chung và một danh mục để kiểm tra thiết kế đã bao phủ các chức năng cần có của một chatbot. Bài không đề cập kiểm soát truy cập, nguồn tài liệu doanh nghiệp hay sinh câu trả lời có căn cứ, nên không thể dùng làm chính thiết kế.

2.2 Framework phân lớp theo hướng repository-first

Boonsri và Buranasaksee (2025) đề xuất một framework chatbot theo hướng repository-first kết hợp với một AI coding agent. Framework dùng kiến trúc Clean Architecture bốn lớp (presentation, application, domain, infrastructure) với các phụ thuộc hướng vào trong. Lớp hạ tầng dùng PostgreSQL với pgvector cho tìm kiếm ngữ nghĩa và Redis cho phiên làm việc và bộ nhớ đệm phân tán. Kênh mới, nguồn dữ liệu mới và nhà cung cấp LLM mới được thêm qua các giao diện adapter nhỏ, nhờ đó có thể đổi nhà cung cấp mà không sửa phần còn lại. Một người review áp dụng checklist nghiệm thu trước khi thay đổi được triển khai.

Các tác giả đánh giá phương pháp bằng thí nghiệm within-subjects với chín lập trình viên, tám microtask tương đương và 144 lượt chạy. So với nhóm chỉ dùng agent, điều kiện dùng framework hoàn thành nhanh hơn khoảng 2,4 lần (tỉ số 2,44; KTC 95% 2,05 đến 3,02) và rẻ hơn khoảng 1,9 lần (tỉ số 1,88; KTC 95% 1,56 đến 2,15), cần ít can thiệp thủ công hơn và tỷ lệ lỗi tương đương (Boonsri & Buranasaksee 2025, Bảng I). Một nghiên cứu riêng với 50 người dùng cho điểm System Usability Scale (SUS) trung bình 70,6, cao hơn mức chuẩn thông dụng 68. Các tác giả thừa nhận hạn chế: nghiên cứu thiếu công suất thống kê, và với tác vụ rất đơn giản, nhóm chỉ dùng agent có thể bằng hoặc tốt hơn. Các con số chính cũng khác nhau giữa phần tóm tắt (nhanh hơn 56%) và Bảng I; tài liệu này dùng số liệu của Bảng I.

2.3 Khoảng trống mà dự án giải quyết

Không công trình nào đề cập bài toán ở đây: trả lời từ tài liệu nội bộ của tổ chức đồng thời bảo đảm ai được đọc gì. Vì vậy dự án kế thừa các ý tưởng kiến trúc có thể áp dụng (phân lớp, adapter, pgvector và bộ nhớ đệm, kiểm thử nghiệm thu có cấu trúc) và bổ sung kiểm soát truy cập tại thời điểm truy xuất, dẫn nguồn, cùng đường từ chối khi không có đoạn liên quan. Bảng 1 cho thấy mỗi công trình đóng góp gì cho thiết kế.

Thành phần thiết kế	Nguồn ý tưởng	Cách dùng trong dự án
Phân tách bốn lớp với phụ thuộc hướng vào trong	Boonsri & Buranasaksee (2025)	Presentation, security, RAG core, dữ liệu và lưu trữ (Mục 4)
Adapter cho LLM, kho dữ liệu và nguồn	Boonsri & Buranasaksee (2025)	Chuyển Ollama (Project A) sang Bedrock (Project B) bằng cấu hình
pgvector và bộ nhớ đệm ở lớp hạ tầng	Boonsri & Buranasaksee (2025)	PostgreSQL với pgvector; semantic cache

Thành phần thiết kế	Nguồn ý tưởng	Cách dùng trong dự án
Checklist nghiệm thu, pass / partial / fail, SUS	Boonsri & Buranasaksee (2025)	Kế hoạch đánh giá (Mục 8)
Theo dõi ngữ cảnh	Mohammed & Aref (n.d.)	Bộ nhớ hội thoại ở bản Beta
Lọc nội dung và xếp hạng trước khi trả lời	Mohammed & Aref (n.d.)	Kiểm tra quyền lần cuối và re-ranker (Beta)
Dự phòng khi các chiến lược trước thất bại	Mohammed & Aref (n.d.)	Trả lời “không tìm thấy”, không gọi LLM
Phân loại ý định của NLU	Mohammed & Aref (n.d.)	Không dùng: mọi câu hỏi đều đi theo đường RAG

Bảng 1. Ánh xạ hai công trình vào thiết kế.

3. Yêu cầu và nguyên tắc thiết kế

3.1 Yêu cầu

Các yêu cầu dưới đây được nêu ở mức cần thiết để dẫn dắt thiết kế. Mã yêu cầu chỉ có hiệu lực trong tài liệu này (R1 đến R8) và không trùng với cách đánh số của Charter.

Mã	Yêu cầu	Bản
R1	Bot chỉ trả lời khi được nhắc tên, trong kênh nằm trong danh sách được duyệt	Alpha
R2	Chỉ người dùng đã đăng ký mới nhận câu trả lời; người khác nhận một lời từ chối ngắn	Alpha
R3	Câu trả lời chỉ được sinh từ các đoạn đã truy xuất và dẫn các trang Confluence đã dùng	Alpha
R4	Khi không có đoạn liên quan, bot nói rõ điều đó và không gọi LLM	Alpha
R5	Câu hỏi và câu trả lời có thể băng tiếng Anh hoặc tiếng Việt; câu trả lời theo ngôn ngữ của câu hỏi	Alpha
R6	Mục tiêu độ chính xác tối thiểu 80% câu trả lời đúng cho mỗi ngôn ngữ và thời gian phản hồi dưới 3 giây	Alpha (mục tiêu)

Mã	Yêu cầu	Bản
R7	Không ai nhận được văn bản vượt mức truy cập được phép, kiểm tra ở bước truy xuất và lại trước khi trả lời	Beta
R8	Mỗi yêu cầu ghi một bản ghi audit với kết quả APPROVED hoặc DENIED	Beta

3.2 Nguyên tắc thiết kế

1. **Mặc định từ chối.** Một yêu cầu chỉ được tiếp tục khi mọi cổng bảo mật đều chấp thuận. Kênh lạ bị bỏ qua, không phản hồi (giả định thiết kế).
2. **Phân quyền trước khi sinh.** Bộ lọc quyền nằm trong chính truy vấn cơ sở dữ liệu, nên văn bản bị hạn chế không bao giờ được truy xuất và không bao giờ đến LLM.
3. **Câu trả lời có căn cứ.** Prompt yêu cầu mô hình chỉ dùng ngữ cảnh được cung cấp và dẫn từng nguồn. Điều này phù hợp với các chiến lược dựa trên tri thức và dựa trên truy xuất trong kiến trúc tham chiếu (Mohammed & Aref n.d.), trong đó bước sinh bị ràng buộc bởi bằng chứng đã truy xuất.
4. **Tách lớp và dùng adapter.** Mỗi lớp chỉ phụ thuộc lớp bên dưới, và các dịch vụ bên ngoài (LLM, kho vector, mô hình embedding, nguồn nội dung) được truy cập qua adapter, như trong Boonsri và Buranasaksee (2025).
5. **Tính năng Beta có thể tắt.** Các thành phần Beta được thiết kế để có thể vô hiệu hóa mà không làm hỏng luồng Alpha.
6. **Một nguồn cấu hình duy nhất.** Token đọc từ biến môi trường. ID kênh, ánh xạ tầng quyền và cài đặt mô hình nằm trong tệp cấu hình, không nằm trong mã nguồn.

4. Kiến trúc hệ thống

Hệ thống được tổ chức thành bốn lớp. Một yêu cầu vào ở Lớp 1, được kiểm tra ở Lớp 2, được trả lời ở Lớp 3 bằng dữ liệu lưu ở Lớp 4, rồi quay về theo đúng đường đó. Lớp 4 còn chạy một đường nạp dữ liệu offline. Sơ đồ kiến trúc một trang và các sơ đồ từng lớp đi kèm tài liệu này dưới dạng hình riêng.

4.1 Các lớp và thành phần

Lớp	Thành phần	Trách nhiệm	Công nghệ chính
1. Presentation	Socket Mode client, event router, payload extractor, Block Kit renderer, Slack Web client	Nhận lượt nhắc tên, dựng gói sự kiện, xác nhận trong 3 giây, hiển thị thẻ trả lời trong thread	Slack Bolt (Python 3.11), Socket Mode (WSS), Block Kit
2. Security	Ingress bridge, whitelist kênh (cổng 1), danh tính (cổng 2), tầng quyền (cổng 3, Beta), egress check, audit logger (Beta)	Quyết định yêu cầu có được tiếp tục không và người hỏi được xem gì	whitelist.yaml, bảng user registry (Project A), đăng nhập một lần của VIB (Project B, giả định)
3. RAG core	Query router, retriever, re-ranker (Beta), generator, bộ nhớ hội thoại (Beta)	Nhận diện ngôn ngữ, dựng bộ lọc quyền, tìm kiếm, xếp hạng lại nếu có, dựng prompt, sinh câu trả lời có dẫn nguồn	Adapter LLM: Ollama (Project A) hoặc Amazon Bedrock (Project B), temperature 0,1
4. Nạp dữ liệu và lưu trữ	Confluence connector, raw staging, format handler, chunker, metadata injector, mô hình embedding, kho vector, semantic cache	Tải, làm sạch, cắt và gắn nhãn trang offline; lưu vector và metadata; phục vụ tìm kiếm tương đồng có lọc và cache	Confluence REST API, PostgreSQL 16 với pgvector (HNSW), metadata JSONB, Redis hoặc cache trong bộ nhớ

Bảng 2. Các lớp, thành phần và công nghệ.

4.2 Tương ứng với kiến trúc tham chiếu

Các lớp ánh xạ vào ba thành phần cốt lõi của Mohammed và Aref (n.d.) như sau. Query router ở Lớp 3 đóng vai trò NLU, được rút gọn thành nhận diện ngôn ngữ và dựng bộ lọc vì không cần phân loại ý định khi mọi câu hỏi đều đi theo một đường truy xuất. Retriever và generator cùng đóng vai trò Dialogue Manager, dùng lần lượt chiến lược dựa trên truy xuất và sinh, với câu trả lời dự phòng cố định thay cho các chiến lược dựa trên luật và tri thức. Giai đoạn phản hồi ở Lớp 2 và Lớp 3 đóng vai trò NLG: egress check là bộ lọc nội dung, còn re-ranker sắp xếp các ứng viên trước khi soạn câu trả lời. Theo dõi ngữ cảnh tương ứng với bộ nhớ hội thoại dự kiến ở Beta.

Chỉ xem cách phân lớp là tương tự về tinh thần với cấu trúc bốn lớp của Boonsri và Buranasaksee (2025), không giống về chi tiết. Các lớp của họ (presentation, application, domain, infrastructure) tách các mối quan tâm phần mềm của một mã nguồn .NET. Các lớp ở đây tách các trách nhiệm lúc chạy của một pipeline truy xuất, với bảo mật được nâng thành một lớp riêng vì kiểm soát truy cập là yêu cầu trung tâm của dự án.

4.3 Giao diện giữa các lớp

- **Lớp 1 sang Lớp 2:** gói sự kiện gồm ID người dùng, ID kênh, mốc thời gian thread, nội dung tin nhắn và email.
- **Lớp 2 sang Lớp 3:** câu hỏi cùng security context (người dùng, email, tầng, phòng ban, ID yêu cầu). Lớp 3 chuyển context này thành bộ lọc cơ sở dữ liệu.
- **Lớp 3 sang Lớp 4:** vector câu hỏi kèm bộ lọc, và tra cache theo khóa gồm câu hỏi và phạm vi quyền (giả định thiết kế).
- **Lớp 3 về Lớp 2 và Lớp 1:** câu trả lời kèm trích dẫn, được Lớp 2 kiểm tra lại trước khi hiển thị.

5. Hành vi khi vận hành

Bảy sơ đồ tuần tự (tổng thể và từng lớp) nằm trong tài liệu đi kèm *Sơ đồ tuần tự: Tổng thể và từng lớp*. Mục này nêu hành vi mà các sơ đồ đó thể hiện.

5.1 Luồng hỏi-đáp

1. Nhân viên nhắc tên bot trong kênh đã duyệt. Slack gửi sự kiện `app_mention` qua WebSocket do bot mở sẵn, và bot xác nhận trong 3 giây vì Slack sẽ gửi lại sự kiện nếu không được xác nhận kịp.
2. Lớp 1 lấy email của người dùng và dựng gói sự kiện. Lớp 2 chạy cổng 1 (kênh), cổng 2 (danh tính) và, ở Beta, cổng 3 (tầng quyền). Bất kỳ cổng nào không đạt đều kết thúc yêu cầu bằng lời từ chối và một bản ghi audit.
3. Lớp 3 nhận diện ngôn ngữ của câu hỏi, dựng bộ lọc quyền từ security context và tra semantic cache. Nếu trúng cache, câu trả lời đã lưu được trả về và bỏ qua các bước còn lại.
4. Nếu không trúng, retriever embedding câu hỏi và yêu cầu 50 đoạn gần nhất thỏa bộ lọc. Ở Beta, một cross-encoder re-ranker thu hẹp còn ba đoạn.
5. Nếu không đoạn nào vượt ngưỡng liên quan, hệ thống trả câu trả lời dự phòng bằng ngôn ngữ của câu hỏi và không gọi LLM.
6. Ngược lại, generator dựng prompt từ các luật cố định, ba đoạn tốt nhất và câu hỏi, gọi LLM, thay dấu trích dẫn bằng liên kết Confluence rồi lưu câu trả lời vào cache. Chỉ câu trả lời có nguồn mới được cache (giả định thiết kế).

7. Lớp 2 kiểm tra mọi tài liệu được trích dẫn nằm trong quyền của người hỏi và ghi bản ghi audit. Lớp 1 đăng tin nhắn Block Kit trong thread, mỗi nguồn có một nút.

Thiết kế có ba lối thoát sớm: từ chối (bước 2), trúng cache (bước 3) và dự phòng (bước 5). Chỉ nhánh cuối mới gọi LLM, nhờ đó các trường hợp phổ biến được xử lý nhanh và rẻ.

5.2 Luồng nạp dữ liệu

1. Quản trị viên hoặc lịch chạy khởi động một lần nạp cho các space key đã chỉ định.
2. Connector duyệt từng trang qua Confluence REST API, thử lại khi bị giới hạn tốc độ và lưu trang thô để có thể chạy lại bộ phân tích mà không gọi thêm.
3. Với mỗi trang, pipeline so sánh checksum MD5 với giá trị đã lưu và bỏ qua trang không đổi.
4. Một format handler chuyển trang thành Markdown sạch. Alpha xử lý trang wiki, bảng phức tạp và liên kết ảnh. Bộ xử lý Word, PDF, Excel và OCR thuộc Beta.
5. Văn bản được cắt thành đoạn 512 token, chồng 50, và gắn nhãn space, phòng ban, loại tệp, mức truy cập, liên kết nguồn và checksum.
6. Các đoạn được embedding và ghi vào kho vector, thay thế các đoạn cũ của cùng trang (giả định thiết kế).

6. Bảo mật và kiểm soát truy cập

Bảo mật được thực thi theo nhiều lớp để một sai sót đơn lẻ không làm lộ văn bản bị hạn chế. Không công trình tham chiếu nào đặc tả kiểm soát truy cập; vì vậy các cơ chế dưới đây là đóng góp riêng của dự án. Chúng chỉ gần với ý tưởng bộ lọc nội dung của Mohammed và Aref (n.d.) ở chỗ một câu trả lời ứng viên được kiểm tra trước khi trả về.

6.1 Các cổng

Cổng	Câu hỏi	Alpha	Beta
1. Whitelist kênh	Kênh này được duyệt chưa?	ID kênh từ whitelist.yaml; kênh lạ bị bỏ qua im lặng (giả định thiết kế)	Như Alpha
2. Danh tính	Người này là ai?	Tra email Slack trong bảng đăng ký do quản trị viên duyệt	Đăng nhập một lần của VIB và kiểm tra miền @vib.com.vn (giả định; Version B chưa được đặc tả)

Cổng	Câu hỏi	Alpha	Beta
3. Tầng quyền	Được xem gì?	Chỉ tầng 1 và 2	Tầng 1 đến 4 quy định mức truy cập tối đa và phòng ban

6.2 Điểm thực thi

- **Khi truy xuất.** Security context trở thành bộ lọc (space, phòng ban, mức truy cập không cao hơn tầng) năm trong cùng truy vấn cơ sở dữ liệu với tìm kiếm vector. Retrieval gateway từ chối mọi lượt tìm không có bộ lọc (giả định thiết kế).
- **Khi trả về (egress).** Trước khi hiển thị, từng tài liệu được trích dẫn được so với tầng của người hỏi. Tài liệu vượt tầng bị loại; nếu không còn gì thì dùng câu trả lời dự phòng.
- **Trong cache.** Khóa cache gồm phạm vi quyền, nên câu trả lời soạn cho một tầng không hiện cho tầng khác (giả định thiết kế).
- **Trong audit log (Beta).** Mỗi yêu cầu ghi một dòng JSON gồm thời gian, email, tầng, câu hỏi, loại tài liệu và kết quả.

6.3 Vai trò Slack và quyền của bot

Vai trò trong workspace Slack (ví dụ Owner hoặc member) chỉ dùng để quản lý workspace và không quyết định bot được tiết lộ gì. Ứng dụng Confluence cài trong Slack chỉ cung cấp xem trước liên kết và thông báo; nó không phải nguồn dữ liệu. Bot đọc Confluence qua REST API bằng API token riêng, lưu trong biến môi trường.

6.4 Ánh xạ tầng quyền

Ánh xạ đề xuất là project manager tầng 4, lập trình viên tầng 2 và khách hàng tầng 1. Đây chỉ là đề xuất và cần khách hàng xác nhận.

7. Nạp dữ liệu và lưu trữ

7.1 Mô hình lưu trữ

Mỗi đoạn được lưu thành một dòng chứa văn bản, vector embedding và một đối tượng metadata JSONB. PostgreSQL với phần mở rộng pgvector lập chỉ mục vector bằng HNSW và lọc theo metadata trong cùng một truy vấn. Thiết kế một kho này theo lựa chọn hạ tầng của Boonsri và Buranasaksee (2025), cũng dùng PostgreSQL với pgvector và bộ nhớ đệm Redis. Dự án ưu tiên cách này vì bộ lọc quyền và xếp hạng tương đồng khi đó chạy trong một truy vấn.

Trường metadata	Mục đích
space_key	Space Confluence mà trang thuộc về
department_tag	Phòng ban được đọc đoạn
file_type	Định dạng nguồn (wiki, bảng, ảnh, và ở Beta là Word, PDF, Excel)
access_level	Tầng tối thiểu cần có để đọc đoạn
source_uri	Liên kết dùng để trích dẫn
checksum_hash	MD5 của nội dung trang, dùng để phát hiện thay đổi

7.2 Quy tắc xử lý

- **Cắt đoạn:** 512 token, chồng 50, để một ý bị cắt ở ranh giới vẫn được tìm thấy.
- **Bảng biểu:** ô gộp hoặc lồng nhau được chuyển đổi để số liệu vẫn gắn với tiêu đề cột.
- **Embedding:** dùng cùng một mô hình khi nạp dữ liệu và khi trả lời, nếu không các vector không so sánh được.
- **Cập nhật:** trang được sửa sẽ thay các đoạn cũ của nó (giả định thiết kế). Trang bị xóa trong Confluence để lại các đoạn mồ côi, có thể dọn bằng kiểm tra định kỳ danh sách ID trang (đề xuất).
- **Lưu tạm:** trang thô được giữ cục bộ ở Project A và trong kho S3 mã hóa ở Project B.

7.3 Bộ nhớ đệm

Semantic cache trả về câu trả lời đã lưu cho một câu hỏi tương tự được hỏi trong cùng phạm vi quyền. Mục trong cache hết hạn sau một khoảng thời gian (giả định thiết kế). Bản nháp của nhóm còn nêu việc cache kết quả tìm kiếm thay vì câu trả lời cuối; cache câu trả lời tiết kiệm cả lệnh gọi LLM, và cần chốt một cách trong decision record.

8. Kế hoạch đánh giá

Cấu trúc đánh giá mượn từ Boonsri và Buranasaksee (2025) và được điều chỉnh cho một hệ thống truy xuất. Đây là kế hoạch, không phải kết quả: chưa có số đo nào được thực hiện.

8.1 Các chỉ số

Chỉ số	Phương pháp	Mục tiêu
Độ đúng của câu trả lời	Bộ câu hỏi kiểm thử cố định cho mỗi ngôn ngữ, hai người chấm pass, partial hoặc fail theo checklist viết sẵn; độ chính xác chuẩn hóa = pass + 0,5 × partial	Tối thiểu 80% cho tiếng Anh và tiếng Việt
Tính hợp lệ của trích dẫn	Mỗi liên kết mở đúng trang hỗ trợ nhận định	Mọi trích dẫn hợp lệ
Hành vi từ chối	Câu hỏi không có đáp án trong kho tài liệu phải nhận câu trả lời dự phòng	Không bịp đặt câu trả lời
Kiểm soát truy cập	Các tài khoản thử ở các tầng khác nhau hỏi cùng một câu; không được xuất hiện văn bản vượt tầng	Không rò rỉ
Thời gian phản hồi	Từ lúc nhắc tên đến lúc đăng câu trả lời, báo riêng trường hợp trúng cache và không trúng	Dưới 3 giây (mục tiêu Alpha)
Khả dụng	Bảng câu hỏi System Usability Scale cho khách hàng và người dùng thí điểm	Trung bình trên 68, ngưỡng của nghiên cứu tham chiếu

8.2 Phương pháp

Boonsri và Buranasaksee (2025) dùng phán đoán pass, partial, fail, độ chính xác chuẩn hóa và khoảng tin cậy bootstrap. Kế hoạch này giữ các thông lệ đó. Vì bộ kiểm thử sẽ nhỏ, kết quả sẽ được báo cáo dưới dạng tỷ lệ kèm khoảng tin cậy thay vì chỉ dùng kiểm định ý nghĩa thống kê, và câu trả lời partial sẽ được tính là thất bại trong chỉ số chính, kèm một phép kiểm tra độ nhạy tính partial là đạt, như trong nghiên cứu tham chiếu.

8.3 Giới hạn của phép so sánh

Nghiên cứu tham chiếu đo năng suất của lập trình viên khi dùng AI coding agent trên các tác vụ LINE và Google Sheets. Nghiên cứu không đánh giá chất lượng truy xuất hay phân quyền ở mức tài liệu, nên các tỉ số thời gian và chi phí của nó không thể dùng để dự đoán hệ thống này. Chỉ các thông lệ đánh giá của nó được tái sử dụng.

9. Rủi ro, hạn chế và các quyết định còn mở

9.1 Rủi ro

Rủi ro	Tác động	Biện pháp giảm thiểu
Gắn sai nhãn quyền khi nạp dữ liệu	Mọi bộ lọc quyền phía sau đều sai	Kiểm thử bước gắn nhãn đầu tiên, với trang mẫu ở từng mức
LLM cục bộ nhỏ trả lời tiếng Việt kém	Không đạt mục tiêu 80% cho tiếng Việt	Thử sớm; so sánh với Bedrock ở Project B
Cache trả câu trả lời sai phạm vi quyền	Rò rỉ thông tin	Đưa phạm vi quyền vào khóa cache; kiểm thử qua các tầng
Giới hạn gói miễn phí của Slack hoặc Confluence	Một số tính năng không dùng được	Xác nhận chi phí gói với bên tài trợ
Phạm vi trượt sang tính năng Beta	Alpha giao trễ	Giữ thành phần Beta có thể tắt và ngoài đường găng của Alpha

9.2 Hạn chế của tài liệu tham khảo

Mohammed và Aref (n.d.) là bài tổng quan mô tả, không có thí nghiệm, bản đã xem không nêu năm công bố hay nơi đăng, và không đề cập LLM hay RAG. Boonsri và Buranasaksee (2025) là bài hội nghị có phản biện, nhưng mẫu nhỏ (chín lập trình viên, tám microtask), công suất thống kê được nêu là thấp hơn mức thông thường, và các con số chưa hoàn toàn nhất quán giữa phần tóm tắt và Bảng I. Cần bổ sung thêm nguồn về RAG và về kiểm soát truy cập trong tìm kiếm doanh nghiệp trước khi nộp báo cáo cuối.

9.3 Các quyết định còn mở

- Số đoạn đưa cho LLM: ba hay năm.
- Mô hình embedding và số chiều vector (ví dụ mô hình được quản lý 1536 chiều hoặc mô hình mã nguồn mở đa ngôn ngữ).
- Kho vector: pgvector hay kho khác; decision record đến hạn ở Tuần 7.
- Lựa chọn LLM cho Project A.
- Thiết kế danh tính cho Project B, hiện chưa được đặc tả.
- Cache lưu câu trả lời cuối hay kết quả truy xuất.
- Xác nhận các giả định thiết kế và ánh xạ tầng quyền với khách hàng.
- Chi phí của các gói Slack hoặc Confluence trả phí.

10. Kết luận

VIB Internal Knowledge Assistant trả lời câu hỏi của nhân viên từ Confluence qua một pipeline bốn lớp, trong đó bảo mật được áp dụng trước khi truy xuất và lại trước khi hiển thị. Kiến trúc tiếp nhận các lựa chọn về phân lớp, adapter và lưu trữ mà Boonsri và Buranasaksee (2025) nêu, và các giai đoạn phản hồi của nó ánh xạ vào các thành phần của kiến trúc tham chiếu trong Mohammed và Aref (n.d.). Điểm riêng của hệ thống là truy xuất có nhận biết phân quyền, trích dẫn bắt buộc và đường từ chối khi không có bằng chứng. Bản Alpha sẽ được đánh giá theo các chỉ số ở Mục 8, và các quyết định còn mở ở Mục 9 phải được thống nhất với khách hàng trước khi sang Beta.

Tài liệu tham khảo

Boonsri, R & Buranasaksee, U 2025, 'A repository-first framework for democratizing chatbot development', *2025 9th International Conference on Information Technology (InCIT)*, IEEE, doi:10.1109/INCIT66780.2025.11276047.

Mohammed, M & Aref, MM n.d., *Chatbot system architecture*, Department of Computer Science, Faculty of Computer and Information Sciences, Ain Shams University, Cairo. [Bản đã xem không nêu năm và nơi công bố; cần xác nhận trước khi nộp.]